

Manual de Instalación y Ejecución

Sistema de Gestión de Citas Médicas

Backend — Express + PostgreSQL 17

Junio 2026

Índice

1. Requisitos del Sistema	3
2. Estructura del Proyecto	3
3. Guía de Instalación Paso a Paso	4
3.1. Paso 1: Instalar PostgreSQL 17	4
3.2. Paso 2: Crear la Base de Datos	4
3.2.1. Opción A: Usando pgAdmin (interfaz gráfica)	4
3.2.2. Opción B: Usando terminal (PowerShell)	4
3.3. Paso 3: Configurar la Conexión	4
3.4. Paso 4: Instalar Dependencias de Node.js	5
3.5. Paso 5: Ejecutar Migraciones	5
3.6. Paso 6: Generar el Cliente Prisma	5
3.7. Paso 7: Poblar la Base de Datos con Datos Iniciales	5
3.8. Paso 8: Migrar Citas de Ejemplo (Opcional)	6
3.9. Paso 9: Iniciar el Servidor	6
4. Probando la API	6
4.1. Verificar que el servidor responde	6
4.2. Obtener especialidades médicas	6
4.3. Listar médicos por especialidad	6
4.4. Ver horarios disponibles	6
4.5. Iniciar sesión	7
4.6. Reservar una cita (requiere token)	7
4.7. Listar citas del paciente	7
5. Resolución de Problemas Comunes	7
5.1. Error: “getaddrinfo ENOTFOUND” al conectar a PostgreSQL	7
5.2. Error: “authentication failed”	7
5.3. Error: “database citas_medicas does not exist”	7
5.4. Error: “port 5432 is already in use”	8
5.5. Error: “Cannot find module” al ejecutar npm run dev	8
6. Estructura de la Base de Datos	8
6.1. Diagrama de Tablas	8
6.2. Descripción de las Tablas	8
7. Script de Instalación Automática	9
8. Comandos Rápidos	10
9. Notas Finales	10

1. Requisitos del Sistema

Antes de comenzar, asegúrese de tener instalado lo siguiente:

1. Node.js versión 18 o superior.

- Descargar de: <https://nodejs.org/>
- Verificar instalación:

```
1 node --version
2 npm --version
3
```

2. PostgreSQL 17.

- Descargar de: <https://www.postgresql.org/download/>
- Durante la instalación, anote la contraseña del usuario postgres.
- Verificar instalación:

```
1 psql --version
2
```

3. Git (opcional, para clonar el repositorio).

- Descargar de: <https://git-scm.com/>

2. Estructura del Proyecto

Al descomprimir el archivo, la estructura del proyecto es la siguiente:

```
1 backend/
2   |-- prisma/
3   |   |-- schema.prisma      # Esquema de la base de datos
4   |   |-- seed.ts           # Datos iniciales (usuarios, doctores)
5   |   |-- migrate-citas.ts  # Migracion de citas.json a BD
6   |   |-- migrations/       # Migraciones SQL generadas
7   |-- src/
8   |   |-- index.ts          # Punto de entrada del servidor
9   |   |-- data/
10  |       |-- database.ts    # Conexion a PostgreSQL con Prisma
11  |       |-- models/        # Interfaces de TypeScript
12  |       |-- services/      # Logica de negocio
13  |       |-- controllers/   # Manejadores HTTP
14  |       |-- middleware/    # Autenticacion
15  |       |-- routes/        # Definicion de rutas Express
16  |       |-- generated/prisma/ # Cliente Prisma generado
17  |-- data/
18  |   |-- citas.json         # Respaldo de datos original (solo
19  |       lectura)
20  |-- package.json
21  |-- tsconfig.json
22  |-- prisma.config.ts      # Configuracion de Prisma CLI
23  |-- .env                  # Variables de entorno (NO incluye
    node_modules)
    node_modules/           # NO INCLUIDO, se genera con npm
    install
```

Nota: La carpeta `node_modules` no se incluye en el archivo comprimido para reducir el tamaño. Se genera automáticamente en el paso `npm install`.

3. Guía de Instalación Paso a Paso

3.1. Paso 1: Instalar PostgreSQL 17

1. Ejecute el instalador de PostgreSQL 17.
2. Acepte todas las opciones por defecto.
3. Cuando solicite una contraseña para el usuario `postgres`, ingrese una que recuerde (ej. `root`).
4. El puerto por defecto es `5432`.
5. Al finalizar, PostgreSQL se ejecutará como un servicio de Windows.

Para verificar que el servicio está corriendo:

```
1 Get-Service postgresql*
```

Debería mostrar estado **Running**. No es necesario tener pgAdmin abierto para que la aplicación funcione.

3.2. Paso 2: Crear la Base de Datos

3.2.1. Opción A: Usando pgAdmin (interfaz gráfica)

1. Abra pgAdmin.
2. Conéctese al servidor local con la contraseña que configuró.
3. Haga clic derecho en **Databases** → **Create** → **Database**.
4. En el campo **Database**, escriba: `citas_medicas`.
5. Haga clic en **Save**.

3.2.2. Opción B: Usando terminal (PowerShell)

```
1 psql -U postgres -c "CREATE DATABASE citas_medicas;"
```

Le pedirá la contraseña del usuario `postgres`.

3.3. Paso 3: Configurar la Conexión

Abra el archivo `backend/.env` y edite la línea con su contraseña real:

```
1 DATABASE_URL="postgresql://postgres:SU_CONTRASENA@localhost:5432/  
citas_medicas"
```

Reemplace `SU_CONTRASENA` por la contraseña que ingresó al instalar PostgreSQL.

3.4. Paso 4: Instalar Dependencias de Node.js

Abra una terminal en la carpeta `backend` y ejecute:

```
1 npm install
```

Esto instalará todas las dependencias necesarias (Express, Prisma, TypeScript, etc.) y generará la carpeta `node_modules`.

3.5. Paso 5: Ejecutar Migraciones

Las migraciones crean automáticamente las tablas en la base de datos:

```
1 npx prisma migrate dev --name init
```

Este comando:

- Lee el archivo `prisma/schema.prisma`.
- Genera el SQL necesario para crear las tablas (`User`, `Doctor`, `Appointment`).
- Aplica las migraciones a la base de datos `citas medicas`.

Si todo funciona correctamente, verá un mensaje como:

```
1 Your database is now in sync with your schema.
```

3.6. Paso 6: Generar el Cliente Prisma

```
1 npx prisma generate
```

Esto genera el cliente ORM tipado en la carpeta `src/generated/prisma/`.

3.7. Paso 7: Poblar la Base de Datos con Datos Iniciales

Ejecute el script de seed para insertar usuarios y doctores de ejemplo:

```
1 npx tsx prisma/seed.ts
```

Debería ver el mensaje:

```
1 Seed completed successfully
```

Los datos insertados son:

- **Usuarios:**
 - Juan Pérez (`juan@email.com`) — Tiene seguro — No penalizado
 - María López (`maria@email.com`) — Sin seguro — No penalizada
 - Carlos Ruiz (`carlos@email.com`) — Tiene seguro — Penalizado
- **Doctores:**
 - Cardiología: Dr. Ana García, Dr. Luis Martínez
 - Dermatología: Dr. Sofía Ramírez, Dr. Pedro Sánchez
 - Pediatría: Dr. Laura Torres, Dr. Andrés Vega

Todas las contraseñas son 123456.

3.8. Paso 8: Migrar Citas de Ejemplo (Opcional)

Si desea cargar las citas de ejemplo del archivo `data/citas.json`:

```
1 npx tsx prisma/migrate-citas.ts
```

Debería ver:

```
1 Migracion completada: 4 citas importadas
```

3.9. Paso 9: Iniciar el Servidor

```
1 npm run dev
```

Este comando usa `nodemon` para reiniciar automáticamente el servidor cuando detecta cambios en el código. Debería ver:

```
1 Servidor corriendo en http://localhost:3000
```

4. Probando la API

Una vez el servidor esté corriendo, puede probar los endpoints. Abra otra terminal (sin cerrar la del servidor).

4.1. Verificar que el servidor responde

```
1 curl http://localhost:3000/api/health
```

Respuesta esperada:

```
1 {"status":"ok","service":"Sistema de Gestion de Citas Medicas"}
```

4.2. Obtener especialidades médicas

```
1 curl http://localhost:3000/api/schedule/especialidades
```

Respuesta esperada:

```
1 ["Cardiologia","Dermatologia","Pediatria"]
```

4.3. Listar médicos por especialidad

```
1 curl http://localhost:3000/api/schedule/medicos/Cardiologia
```

4.4. Ver horarios disponibles

```
1 curl "http://localhost:3000/api/schedule/horarios?medicoId=2&fecha=2026-06-26"
```

4.5. Iniciar sesión

```
1 curl -X POST http://localhost:3000/api/auth/login ^
2   -H "Content-Type: application/json" ^
3   -d "{\"email\":\"juan@email.com\",\"password\":\"123456\"}"
```

Respuesta esperada (devuelve un token y datos del usuario):

```
1 {"token":"...", "user":{"id":"1", "nombre":"Juan Perez", "email":"
  juan@email.com", ...}}
```

4.6. Reservar una cita (requiere token)

```
1 curl -X POST http://localhost:3000/api/appointments/reservar ^
2   -H "Content-Type: application/json" ^
3   -H "Authorization: Bearer TOKEN_AQUI" ^
4   -d "{\"medicoId\":\"2\",\"especialidad\":\"Cardiologia\",\"fecha\":\"
  2026-07-01\",\"hora\":\"10:00\"}"
```

4.7. Listar citas del paciente

```
1 curl http://localhost:3000/api/appointments ^
2   -H "Authorization: Bearer TOKEN_AQUI"
```

5. Resolución de Problemas Comunes

5.1. Error: “getaddrinfo ENOTFOUND” al conectar a PostgreSQL

- Verifique que el servicio de PostgreSQL esté corriendo:

```
1 Get-Service postgresql*
2
```

- Si no está corriendo, inícielo:

```
1 Start-Service postgresql-x64-17
2
```

5.2. Error: “authentication failed”

- Revise que la contraseña en `.env` coincida con la que configuró en PostgreSQL.

5.3. Error: “database citas_medicas does not exist”

- Aún no ha creado la base de datos. Vuelva a la sección 3.2.

5.4. Error: “port 5432 is already in use”

- Otra instancia de PostgreSQL ya está usando el puerto 5432.
- Puede cambiar el puerto en el archivo `.env` agregando `?port=5433` al final de la URL.

5.5. Error: “Cannot find module” al ejecutar `npm run dev`

- Ejecute `npm install` nuevamente para regenerar `node_modules`.

6. Estructura de la Base de Datos

6.1. Diagrama de Tablas

+-----+			+-----+			+-----+		
User			Appointment			Doctor		
+-----+			+-----+			+-----+		
id			pacienteId			id		
nombre			medicoId			nombre		
email			especialidad			especial		
pass			fecha			+-----+		
seguro			hora					
penali			estado					
+-----+			descuento					
			createdAt					
			+-----+					

6.2. Descripción de las Tablas

User: Almacena los pacientes del sistema.

- `id`: Identificador único.
- `nombre`: Nombre completo del paciente.
- `email`: Correo electrónico (único).
- `password`: Contraseña de acceso.
- `tieneSeguro`: Indica si el paciente tiene seguro médico.
- `penalizado`: Indica si el paciente tiene bloqueos administrativos.

Doctor: Almacena los médicos disponibles.

- `id`: Identificador único.
- `nombre`: Nombre completo del médico.
- `especialidad`: Especialidad médica.

Appointment: Almacena las citas médicas.

- **id:** Identificador único (formato: CITA- + timestamp).
- **pacienteId:** Referencia al paciente (relación con User).
- **medicoId:** Referencia al médico (relación con Doctor).
- **especialidad:** Especialidad de la cita.
- **fecha:** Fecha de la cita (formato YYYY-MM-DD).
- **hora:** Hora de la cita (formato HH:MM).
- **estado:** Estado de la cita (pendiente, confirmada, cancelada, completada).
- **descuentoAplicado:** Indica si se aplicó descuento por seguro.
- **createdAt:** Fecha y hora de creación del registro.

7. Script de Instalación Automática

Para facilitar la instalación en otra máquina, incluya el siguiente archivo `setup.bat` en la raíz del proyecto `backend`:

```
1 @echo off
2 echo =====
3 echo  Configuración del Sistema de Citas
4 echo  =====
5 echo .
6
7 echo [1/5] Instalando dependencias...
8 call npm install
9 if %errorlevel% neq 0 (
10     echo Error instalando dependencias
11     pause
12     exit /b 1
13 )
14
15 echo [2/5] Ejecutando migraciones...
16 call npx prisma migrate dev --name init
17 if %errorlevel% neq 0 (
18     echo Error ejecutando migraciones
19     pause
20     exit /b 1
21 )
22
23 echo [3/5] Generando cliente Prisma...
24 call npx prisma generate
25 if %errorlevel% neq 0 (
26     echo Error generando cliente
27     pause
28     exit /b 1
29 )
30
31 echo [4/5] Insertando datos iniciales...
32 call npx tsx prisma/seed.ts
```

```

33 if %errorlevel% neq 0 (
34     echo Error insertando datos
35     pause
36     exit /b 1
37 )
38
39 echo [5/5] Migrando citas de ejemplo...
40 call npx tsx prisma/migrate-citas.ts
41
42 echo.
43 echo =====
44 echo Instalacion completada con exito
45 echo =====
46 echo.
47 echo Ejecute 'npm run dev' para iniciar el servidor
48 echo.
49 pause

```

8. Comandos Rápidos

Comando	Descripción
<code>npm install</code>	Instalar dependencias
<code>npm run dev</code>	Iniciar servidor en modo desarrollo
<code>npx prisma studio</code>	Abrir interfaz gráfica de la BD
<code>npx tsx prisma/seed.ts</code>	Poblar datos iniciales
<code>npx tsx prisma/migrate-citas.ts</code>	Importar citas de ejemplo
<code>npx tsc --noEmit</code>	Verificar errores de TypeScript

9. Notas Finales

- La carpeta `node_modules` no se incluye en el archivo comprimido. Siempre ejecute `npm install` después de descomprimir.
- La base de datos no viaja en el proyecto. Debe crearla localmente siguiendo los pasos de la sección 3.
- El archivo `data/citas.json` se mantiene como respaldo de solo lectura. Los datos reales están en PostgreSQL.
- Si usa Windows PowerShell, reemplace `^` por `'` al hacer saltos de línea en los comandos `curl`.
- Para apagar el servidor, presione `Ctrl + C` en la terminal donde se ejecuta.